

Linux I/O Architectures Evolution

<https://gemini.google.com/app/a07df9d722fc692e>

User prompt: 7. Restaurant Analogy for Linux I/O Architectures Here is how the evolution of Linux I/O architectures translates to operating a busy restaurant with 1,000 customers (sockets).

7.1 `select()` (1983): The Flawed Paper Survey Imagine a restaurant manager running a 1,000-table dining hall with a single waiter: ``text [WAITER] --> Hands out 1,000 clipboards to tables 1-1000 [WAITER] --> Walks to Table 1, 2, 3... 1000 checking every box [WAITER] --> Finds Table 4 wants water and crosses out the other boxes [WAITER] --> Delivers water to Table 4 [REPEAT] --> Prints and hands out 1,000 new clipboards for the next loop `` How it works: you hand the waiter a pre-printed survey form listing all 1,000 tables. The waiter walks to every table from #1 to #1000 and checks whether anyone wants to order. The flaw: when Table 4 says, "I want water," the waiter modifies the set in place. The kernel returns only the ready entries, so the application must rebuild the complete descriptor set before the next call. Why it does not scale: 1. **Destructive state:** because the descriptor sets are modified during the call, the application must prepare them again for the next loop. 2. **The 1,024 cap:** the traditional `FD_SETSIZE` limit commonly restricts the descriptors that can be represented by `fd_set`. 3. **Heavy scanning:** even if 999 tables are quiet, the waiter checks all of them on every pass. ### 7.2 `poll()` (1989): The Unlimited Reusable Array You decide that reprinting paper surveys is wasteful, so you buy reusable wooden order blocks, one for each table: ``text [ WAITER ] --> Scans wooden blocks for Table 1 to 1000 sequentially [ WAITER ] --> Table 4 flips its block to READY [ WAITER ] --> Resets Table 4's block to IDLE [ WAITER ] --> Delivers water to Table 4 `` How it works: the application passes an array of `pollfd` structures. Each entry contains a file descriptor and the events of interest. The kernel updates the returned event fields, and the application scans the array for ready entries. What it solved: the fixed `FD_SETSIZE` representation limit is removed. The array can contain many descriptors, subject to memory and system limits. The application also avoids the bit-set manipulation used by `select()`. The remaining bottleneck is still $O(N)$ scanning. The waiter walks past all 1,000 tables on every pass, even when only one table is active. The array also has to be supplied to the kernel on each wait operation. ### 7.3 `epoll` (2002): The Kitchen Service Bell You fire the walking waiter and install a centralized notification bell at the kitchen counter: ``text [ KITCHEN ] --> Maintains a master ledger of registered tables [ TABLE 4 ] --> Rings its table bell [ KITCHEN ] --> Places Table 4 on a ready list [ WAITER ] --> Checks only the ready list [ WAITER ] --> Walks to Table 4 and performs the I/O `` How it works: 1. A file descriptor is registered once with `epoll_ctl` in the kernel's interest list. 2. When an event occurs, the kernel places the ready descriptor in an internal ready list. 3. The application calls `epoll_wait` and receives only descriptors that are ready instead of scanning every registered descriptor. What it solved: idle descriptors are not repeatedly scanned by user space. When 999 tables are quiet, the waiter can sleep until the ready list contains an active table. This gives efficient readiness notification, especially when only a small portion of the descriptor set is active. The remaining cost is that readiness is not completion. After `epoll_wait` reports a readable descriptor, the application must still call `read` or `recv`. Sending a response requires another `write` or `send` operation. Conceptually, there is one wait step followed by one or more I/O steps. ### 7.4 `io\_uring` (2019): The Shared Counter and Pass-Through Window You eliminate most of the waiter's back-and-forth movement by installing a two-sided shared counter between user space and kernel space: ``text
COUNTER (Shared RAM) | +-----+ | THE
Queue (SQ)] ----->+ [Submission
Kernel places orders | | User ----->+ [Submission
| User <-----+ [Completion Queue (CQ)] <-----+ Kernel reads results
| | writes results +-----+ `` How it works:
1. **Submission Queue (SQ):** the application places operations such as "receive up to 4 KB from client 4" into shared queue entries. 2. **Kernel execution:** the kernel consumes submitted operations and executes them. Multiple operations can be submitted together, which enables batching. 3. **Completion Queue (CQ):** when an operation finishes, the kernel writes a completion event into the shared completion queue. The application reads the result and

dispatches the next operation. The application submits work in advance instead of first waiting for readiness and then issuing a separate I/O call. The exact syscall reduction depends on how the ring is used. This project uses raw ``io_uring_setup`` and ``io_uring_enter`` calls and does not depend on ``liburing``. shorten for ppt

Response: **Slide 1: `select()` (1983) – The Flawed Paper Survey**

- **Analogy:** A waiter hands paper survey forms to 1,000 tables, checks every box sequentially, modifies the form, and reprints 1,000 new forms for the next round.
- **Mechanism:** Passes fixed bitmasks (`fd_set`); kernel modifies the mask in-place to reflect active descriptors.
- **Key Drawbacks:**
 - **Destructive State:** Application must rebuild descriptor sets before every call.
 - **Hard Limit:** Restricted to 1,024 file descriptors (`FD_SETSIZE`).
 - $O(N)$ **Scanning:** Forces full linear checks across all descriptors every loop.

Slide 2: `poll()` (1989) – The Unlimited Reusable Array

- **Analogy:** Tables use reusable wooden status blocks. The waiter still walks past all 1,000 tables sequentially to inspect each block.
- **Mechanism:** Passes an array of `pollfd` structures detailing file descriptors and target events.
- **Key Improvements:** Eliminates the hard-coded 1,024 descriptor limit.
- **Remaining Bottleneck:**
 - $O(N)$ **Scanning:** Still scans every descriptor on every wait operation.
 - **Memory Overhead:** The full descriptor array must be re-sent to the kernel each time.

Slide 3: `epoll` (2002) – The Kitchen Service Bell

- **Analogy:** Active tables ring a kitchen bell. The kitchen logs active tables on a central board, so the waiter only visits tables listed on the board.
- **Mechanism:**
 1. File descriptors register once with `epoll_ctl`.
 2. Kernel puts active descriptors onto an internal ready list.
 3. `epoll_wait` returns **only** ready descriptors.
- **Key Improvements:** $O(1)$ readiness notification; idle sockets consume zero CPU scan time.
- **Remaining Bottleneck:** Readiness is not completion. Applications still require separate syscalls (`read/write`) after receiving notification.

Slide 4: `io_uring` (2019) – Shared Memory Ring Buffers

- **Analogy:** A two-sided counter window. Customers drop written order slips into a submission box; kitchen staff fulfill them and place finished orders into a completion box.
 - **Mechanism:**
 - **Submission Queue (SQ):** Application batches I/O operations directly into shared RAM.
 - **Completion Queue (CQ):** Kernel executes I/O asynchronously and posts results to shared RAM.
 - **Key Improvements:**
 - **Asynchronous Completion:** Merges notification and execution into one paradigm.
 - **Syscall Reduction:** Reduces context switches via shared ring buffers between user and kernel space.
-

User prompt: how include in this slide

Response: I will now generate a slide deck incorporating the restaurant analogy into the Linux I/O architecture framework, styled to match the design from your uploaded diagram.

Your presentation deck on the **Restaurant Analogy for Linux I/O Architectures** is ready! It integrates the five-engine design structure from your reference diagram with detailed slides for `select`, `poll`, `epoll`, and `io_uring`.

Feel free to review the slides and let me know if you would like to adjust any formatting, colors, or text details!

User prompt: i dont want entire slide deck i want to include the analogy in just this one slide

Response: I will now generate the single slide deck that integrates the restaurant analogy directly into the 5-engine architecture layout from your image.

I've updated the presentation into a single, comprehensive slide that merges the exact visual layout from your uploaded image with the 1,000-table restaurant analogies inside each card. Feel free to review it and let me know if you would like any further adjustments!
